

Informe técnico — SentinelDrive (SENTINLDRIVE)

Fecha: 26 de marzo de 2026

Repositorio local: C:\Users\TheterAlien\Desktop\sentinelDrive

Alcance: descripción de tecnologías, arquitectura, prácticas, bases de datos, datos recolectados y funcionalidades **según el código y configuración presentes en este proyecto.**

Nota de seguridad: este informe **no incluye** valores sensibles (claves/contraseñas).

1) Resumen del proyecto

SentinelDrive (SENTINLDRIVE) es una aplicación web para **monitoreo y gestión de flota vehicular** (buses) con:

- **Ingesta IoT por MQTT** (telemetría GPS + eventos críticos).
- **Persistencia en base de datos** (por defecto SQLite) para historial, reportes y auditoría operativa.
- **Panel web** con autenticación y roles (admin/user) para:
 - Dashboard de estado del sistema y salud de buses.
 - Seguimiento de última posición en mapa.
 - Gestión de eventos críticos y notificaciones.
 - Reportes estadísticos con gráficas y exportación a CSV.
 - Gestión administrativa (buses, usuarios, mantenimientos, purga de historial).

2) Tecnologías utilizadas (stack)

2.1 Backend (Python / Flask)

Dependencias principales (según `requirements.txt`):

- **Python 3.12** (entorno venv/).
- **Flask 3.1.2:** servidor web + enrutamiento + sesión.
- **Jinja2 3.1.6:** plantillas HTML.
- **Flask-SQLAlchemy 3.1.1 + SQLAlchemy 2.0.44:** ORM y acceso a base de datos.
- **Flask-WTF 1.2.2 + CSRFProtect:** protección CSRF en formularios.
- **python-dotenv 1.2.1:** carga de variables de entorno desde `.env`.
- **paho-mqtt 2.1.0:** cliente MQTT (suscripción y publicación).
- **Werkzeug 3.1.3:** utilidades web (y hashing de contraseñas en el modelo de usuario).
- **gunicorn 21.2.0:** servidor WSGI para despliegues (típicamente Linux).

2.2 Frontend (UI)

Tecnologías de interfaz:

- **Bootstrap 5.3.3** (CDN) + **Bootstrap Icons**: layout, componentes y estilo.
- **JavaScript** propio para:
- Toggle de sidebar y UX general.
- **Notificaciones** (toasts) mediante polling de eventos.
- **Leaflet 1.9.4** + **OpenStreetMap**: mapas para seguimiento GPS.
- **Chart.js** (CDN): gráficas en reportes.

2.3 IoT / Mensajería (MQTT)

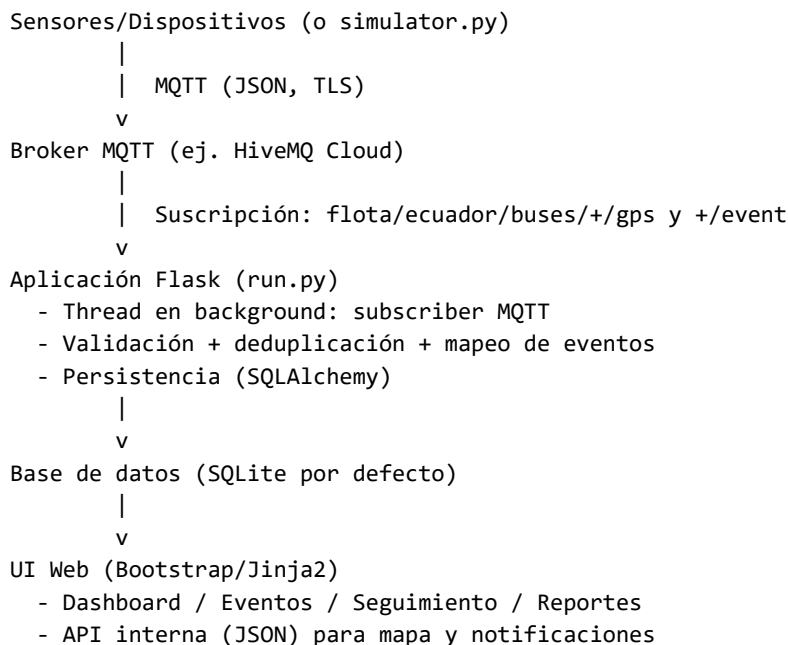
- **Broker MQTT** configurable por variables de entorno.
- Conexión con **TLS** (puerto típico 8883).
- Suscripción a tópicos con comodín para flota:
- flota/ecuador/buses/+/gps (QoS 0)
- flota/ecuador/buses/+/event (QoS 1)

2.4 Base de datos

- **Tipo**: relacional (SQL) vía SQLAlchemy.
- **Implementación por defecto**: **SQLite** (archivo instance/sentinel drive.db).
- **Configurable**: DATABASE_URL permite apuntar a otros motores soportados por SQLAlchemy (requiriendo el driver correspondiente).

3) Arquitectura (vista general)

3.1 Flujo de datos (alto nivel)



3.2 Estructura del proyecto (módulos)

- `run.py`: arranque del servidor Flask + inicialización de datos + inicio del suscriptor MQTT en segundo plano.
- `app/__init__.py`: **Application Factory** (`create_app`) + registro de blueprints + `db.create_all()` + creación de índices.
- `app/config.py`: configuración por variables de entorno (secretos, DB, MQTT, cookies de sesión, CSRF, debug).
- `app/extensions.py`: inicialización centralizada de extensiones (`db`, `csrf`).
- `app/models/*`: modelos ORM (`Bus`, `Location`, `Event`, `Maintenance`, `User`) + inicialización/índices.
- `app/routes/*`: blueprints (`auth`, `dashboard`, `tracking`, `events`, `reports`, `buses`, `users`, `maintenance`, `admin`).
- `app/mqtt/*`: suscriptor MQTT + deduplicación de mensajes.
- `app/templates/*`: páginas HTML.
- `static/*`: CSS/JS/imagenes.
- `instance/sentinldrive.db`: base de datos local (SQLite).
- `simulator.py`: simulador IoT (publica GPS y eventos al broker).

3.3 Patrones y decisiones de diseño observadas

- **Monolito web** (Flask) con **ingesta IoT** embebida vía thread.
- **Blueprints** para modularidad de rutas.
- **ORM** con relaciones y cascadas para consistencia referencial.
- **Event-driven** en ingreso de datos (pub/sub MQTT).
- **Timestamps con zona horaria** (America/Guayaquil) para coherencia temporal.
- **Deduplicación en memoria** para evitar escrituras repetidas por reintentos o duplicados cercanos.

4) Bases de datos: modelo y tablas

4.1 Entidades principales

- **Bus** (bus)
 - `id` (PK), `plate` (única), `driver`, `status` (por defecto "Activo"), `description` (opcional)
- Relaciones:
 - 1:N con **Location**, **Event**, **Maintenance**
 - Cascada delete-orphan (al eliminar un bus se elimina su historial asociado)
- **Location** (location) — posiciones GPS
 - `id` (PK), `bus_id` (FK), `lat`, `lon`, `speed`, `timestamp`
 - Índice: (`bus_id`, `timestamp`)
- **Event** (event) — eventos críticos

- `id` (PK), `bus_id` (FK), `type`, `value` (opcional), `description` (opcional), `latitude`, `longitude`, `timestamp`
- Índice: (`bus_id`, `timestamp`)
- **Maintenance** (`maintenance`) — mantenimientos
- `id` (PK), `bus_id` (FK), `description`, `date`, `status` (por defecto “Pendiente”)
- Índice: (`bus_id`, `date`)
- **User** (`user`) — usuarios del panel
- `id` (PK), `username` (único), `password_hash`, `role` (`admin/user`)

4.2 Gestión de esquema

- Las tablas se crean con `db.create_all()` al iniciar la app (sin migraciones formales).
- Se crean índices “si no existen”.
- Para SQLite se observa una evolución “ligera” del esquema mediante `ALTER TABLE` para columnas opcionales (sin romper BD existentes).

5) Datos recolectados (telemetría y operación)

5.1 Telemetría GPS (MQTT → Location)

Campos típicos en mensajes gps:

- Identificación: `bus_id`
- Tiempo: `timestamp` (ISO 8601)
- Posición: `lat`, `lon`
- Velocidad: `speed_gps` (o `speed`)

Persistencia:

- Se almacena una fila por recepción en `location` (con `timestamp`).

5.2 Eventos críticos (MQTT → Event)

El suscriptor mapea códigos de evento a tipos legibles:

- `exceso_velocidad` → **Exceso de velocidad**
- `frenado_brusco` → **Frenado brusco**
- `curva_peligrosa` → **Curva pronunciada**
- `conduccion_agresiva` → **Conducción agresiva**
- `sobrecalentamiento` → **Sobrecalentamiento**
- `otros/otro` → **Otros** (sensores extendidos)

Campos típicos:

- `bus_id`, `timestamp`, `event` (código)
- Coordenadas: `lat`, `lon` (si vienen en el payload)

- Valor del evento (se guarda en `event.value`):
- velocidad (OBD/GPS), aceleración, RPM, temperatura, etc. según el evento
- Para **Otros**:
- `description` (qué sensor/alerta es)
- `value` (valor numérico asociado)

Controles de calidad observados:

- Validación básica de `bus_id`, `timestamp` y coordenadas.
- Sanitización de `description` (recorta longitud máxima).
- **Deduplicación temporal** en memoria (TTL) para reducir duplicados cercanos.

5.3 Datos operativos (panel)

Además de telemetría, el sistema almacena y gestiona:

- **Usuarios y roles** (admin/user).
- **Catálogo de buses** (placa, conductor, estado, descripción).
- **Mantenimientos** (descripción, fecha, estado).
- Auditoría por logs para acciones críticas (p. ej. purga de historial con IP y usuario).

6) Funcionalidades del sistema (por módulos)

6.1 Autenticación y autorización

- Login en `/` y logout por POST.
- Sesión server-side (cookie de sesión) con:
- `session["user"]` y `session["role"]`.
- Rutas protegidas con decoradores:
- `login_required` (requiere sesión).
- `require_admin` (requiere rol admin).

6.2 Dashboard

- Métricas rápidas: total de buses, activos, eventos, mantenimientos pendientes.
- “Salud” por bus basada en *último GPS recibido* (ej. conectado si recibió en los últimos ~60s).
- Estado del MQTT: conexión, última conexión, último mensaje, etc.

6.3 Seguimiento (Tracking)

- Vista `/tracking` con:
- Selector de bus.
- Mapa Leaflet (OpenStreetMap).
- Actualización automática (polling) para mostrar la **última posición**.
- API: `/api/last-position/<bus_id>` (JSON).

6.4 Eventos

- Vista /events (paginación).
- API: /api/events/updates para notificaciones (polling).
- UI: toasts + badge de nuevos eventos.

6.5 Reportes

- Vista /reports con filtros:
- Periodo: día, semana, mes, 6 meses, año.
- Flota completa o bus individual.
- Estadísticas:
- Eventos por tipo y por hora.
- Velocidad: media/máx/mín.
- Histograma de velocidad (bins de 10 km/h).
- Puntaje de riesgo (ponderación simple basada en tipos de eventos).
- Descargas:
- CSV de eventos del periodo.
- CSV de tablas seleccionadas (bus/event/maintenance/location).

6.6 Administración (solo admin)

- Gestión de buses:
- Alta/edición/borrado.
- Exportación CSV del catálogo.
- Gestión de mantenimientos:
- Registro, cambio de estado (pendiente/completado), eliminación.
- Gestión de usuarios:
- Crear, cambiar rol, cambiar contraseña, eliminar (protege admin principal).
- Panel admin:
- **Purga de historial** (Location + Event) con reautenticación y logs de auditoría.

6.7 Simulador IoT

simulator.py publica datos de ejemplo al broker MQTT, incluyendo:

- GPS periódico.
- Generación probabilística de eventos (exceso de velocidad, frenado, curvas, agresividad, temperatura, otros sensores).

Útil para:

- Demostraciones.
- Pruebas de UI/reportes sin hardware real.

7) Prácticas y consideraciones relevantes

7.1 Configuración por entorno

La app toma configuración desde `.env` (cargada con `python-dotenv`). Variables destacadas:

- `SECRET_KEY` (obligatoria).
- `DATABASE_URL` (por defecto SQLite).
- `MQTT_BROKER`, `MQTT_PORT`, `MQTT_USERNAME`, `MQTT_PASSWORD` (`MQTT_PASSWORD` es obligatoria).
- `DEFAULT_ADMIN_PASSWORD` (para bootstrap del usuario admin si no existe).
- Flags: `FLASK_DEBUG`, `WTF_CSRF_ENABLED`, `SESSION_COOKIE_*`, `SQLALCHEMY_ECHO`.

Recomendación: mantener secretos fuera de control de versiones, rotarlos y usar valores fuertes.

7.2 Seguridad

- Hash de contraseñas (no se guardan contraseñas en texto plano).
- CSRF habilitado para formularios.
- Control de acceso por rol (admin/user).
- Cookies de sesión con flags configurables (`HttpOnly`, `SameSite`, `Secure`).

7.3 Observabilidad y operación

- Logging centralizado (formato consistente).
- Estado del MQTT expuesto al dashboard (conectado, última conexión, última actividad, error).
- Mecanismo de reconexión MQTT con backoff.

7.4 Rendimiento básico

- Índices sobre timestamp por bus para consultas frecuentes.
- Paginación y límites en listados (eventos).
- Deduplicación en memoria para reducir escritura redundante.

8) Limitaciones actuales y mejoras sugeridas (corto)

- **Migraciones:** incorporar Alembic/Flask-Migrate para cambios de esquema más robustos.
- **Escalabilidad:** separar ingesta MQTT del servidor web (worker/servicio aparte) si crece la carga.
- **Drivers de BD:** si se migra a Postgres/MySQL, agregar el driver correspondiente y ajustar configuración.
- **Pruebas:** añadir pruebas unitarias/funcionales mínimas (ingesta MQTT, rutas críticas, reportes).

- **Gestión de secretos:** reemplazar `.env` en repos por `.env.example` y secret manager en producción.

Anexo A) Ejemplos de payload MQTT (referenciales)

GPS

```
{
  "bus_id": 1,
  "type": "gps",
  "timestamp": "2026-03-26T12:34:56-05:00",
  "lat": -4.000123,
  "lon": -79.200456,
  "speed_gps": 42.3
}
```

Evento: frenado brusco

```
{
  "bus_id": 1,
  "type": "event",
  "event": "frenado_brusco",
  "timestamp": "2026-03-26T12:35:10-05:00",
  "accel_x": -5.2,
  "lat": -4.000120,
  "lon": -79.200430
}
```

Evento: otros (sensores extendidos)

```
{
  "bus_id": 1,
  "type": "event",
  "event": "otros",
  "timestamp": "2026-03-26T12:36:00-05:00",
  "description": "Presion de llantas (psi)",
  "value": 32.5,
  "lat": -4.000110,
  "lon": -79.200410
}
```